

Multi-Algorithm Puzzle Solver: A Unified Artificial Intelligence Framework for Search-Based Problem Solving

Abstract

This report presents the design and development of a unified Artificial Intelligence framework for solving multiple puzzle environments using classical and heuristic search techniques. The proposed system integrates 8-Puzzle, Maze Solver, and N-Queens problems into a common state-space representation.

The framework implements Breadth First Search (BFS), Depth First Search (DFS), Uniform Cost Search (UCS), Greedy Best First Search, and A* Search. Unlike a conventional implementation that focuses on a single algorithm, this project provides a comparative environment where users can observe algorithm behaviour, visualize solution generation, and analyse computational performance.

The system uses a Streamlit-based interface that connects user interaction with the underlying AI search engine. Performance is evaluated using runtime, nodes expanded, nodes generated, frontier size, and path cost.

Keywords: Artificial Intelligence, State Space Search, BFS, DFS, UCS, A*, Heuristic Search, Visualization.

1. Introduction and Motivation

Artificial Intelligence systems frequently solve problems by exploring possible states and selecting appropriate actions to achieve a target objective. Search-based problem solving is a fundamental AI technique used in robotics, navigation, planning, and automated decision systems.

The major challenge in search problems is the rapid growth of possible states. A simple problem can generate thousands or millions of possible configurations, making inefficient exploration expensive in terms of time and memory.

Puzzle environments provide an effective platform for analysing AI search because they contain clearly defined initial states, available actions, transition rules, and goal conditions. This project transforms theoretical search concepts into a practical framework where different algorithms can be studied and compared.

The objectives of this project are:

- Develop a unified multi-puzzle solving framework.
- Implement uninformed and informed search algorithms.
- Provide visual interpretation of algorithm behaviour.
- Evaluate computational efficiency using measurable indicators.

2. Proposed System Architecture

The proposed system follows a modular architecture where each component performs a specific responsibility. The separation of interface, computation, and evaluation improves maintainability and future scalability.

The workflow begins when a user selects a puzzle environment and algorithm through the Streamlit interface. The selected puzzle is converted into a mathematical state representation. The search engine explores possible states and generates a solution path. The verification module confirms correctness, while visualization and evaluation modules analyse the final output.

Advantages of the architecture:

- Supports multiple puzzle environments.
- Allows independent testing of algorithms.
- Simplifies future extension.
- Provides clear understanding of system operation.

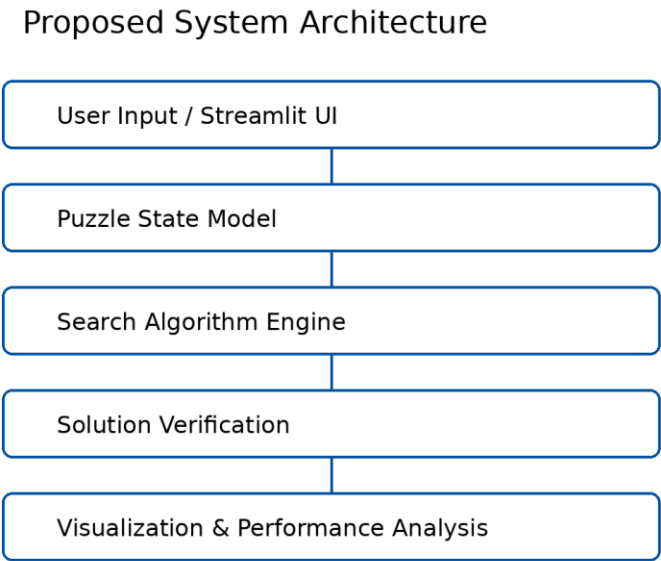


Figure 1. Proposed System Architecture

3. Mathematical Representation and Puzzle Modelling

A search problem can be represented as:

$$P = (S, A, T, C, G)$$

where:

S represents the state space.

A represents available actions.

T represents state transition behaviour.

C represents path cost.

G represents the goal state.

For informed search:

$$f(n) = g(n) + h(n)$$

where $g(n)$ represents accumulated path cost and $h(n)$ represents estimated remaining cost.

Puzzle modelling:

8-Puzzle:

The board is represented as tile positions. Every valid movement generates a new state until the desired configuration is reached.

Maze Solver:

The environment is represented as a grid where each accessible location becomes a possible state.

N-Queens:

The problem is represented as a constraint satisfaction problem where queens must be placed without conflicts.

4. Search Algorithm Methodology

The framework implements both uninformed and informed search techniques.

Breadth First Search (BFS):

BFS explores nodes level-by-level using a queue. It provides optimal solutions when all transition costs are equal, but its memory requirement can increase significantly.

Depth First Search (DFS):

DFS explores deeper states before backtracking. It requires less memory but does not guarantee optimal solutions.

Uniform Cost Search (UCS):

UCS expands states according to accumulated path cost and is effective for variable-cost environments.

Greedy Best First Search:

Greedy search uses heuristic information to select states that appear closer to the goal.

A* Search:

A* combines actual cost and heuristic estimation using $f(n)=g(n)+h(n)$. This provides an efficient balance between exploration and solution quality.

5. Heuristic Analysis, Software Implementation and Visualization

Heuristics provide additional information to informed search algorithms by estimating the distance between the current state and the goal.

For grid-based environments, Manhattan distance is commonly used:

$$h(n) = \Sigma(|x_{\text{current}} - x_{\text{goal}}| + |y_{\text{current}} - y_{\text{goal}}|)$$

A good heuristic reduces unnecessary exploration while maintaining solution reliability.

Software structure:

app.py:

Controls interface operation and user interaction.

core/puzzles:

Defines states and possible actions.

core/search:

Contains algorithm implementations.

core/heuristics:

Provides heuristic calculations.

Visualization:

Displays puzzle states, routes, and final solutions to improve user understanding.

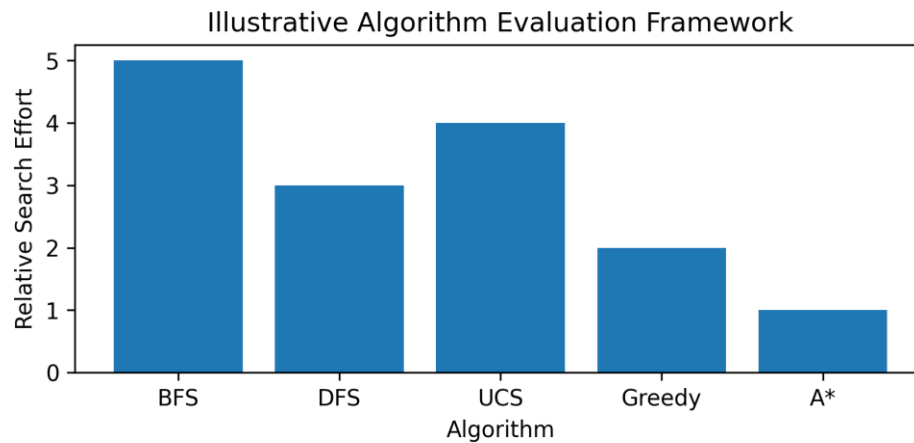


Figure 2. Illustrative Algorithm Evaluation Framework

6. Performance Evaluation Framework

The performance evaluation framework analyses algorithms beyond simply checking whether a solution exists. Different algorithms may achieve the same goal while requiring different computational resources.

Evaluation metrics:

Runtime:

Measures execution speed.

Nodes Expanded:

Represents the number of states examined.

Nodes Generated:

Shows search complexity.

Maximum Frontier Size:

Indicates memory requirements.

Path Cost:

Represents solution efficiency.

The evaluation framework helps compare optimality, computational cost, and resource consumption.

An illustrative comparison framework is shown below.

7. Results, Discussion, Conclusion and References

Results and Discussion:

Different algorithms demonstrate different strengths. BFS provides completeness and optimality but requires higher memory. DFS reduces memory usage but may sacrifice solution quality. UCS performs effectively when path costs vary. Greedy approaches can reach solutions quickly but depend on heuristic accuracy. A* provides a balance by combining actual and estimated cost.

Conclusion:

This project successfully develops a complete AI-based puzzle solver integrating multiple search algorithms, visualization, and performance analysis.

Future Work:

Future improvements include additional puzzle environments, improved animations, learning-based search strategies, and online deployment.

References:

- [1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach.
- [2] P. E. Hart, N. J. Nilsson, and B. Raphael, A Formal Basis for the Heuristic Determination of Minimum Cost Paths.
- [3] Streamlit Documentation.
- [4] Python Documentation.